

Exam Resilience Control Tower (ERCT)

Product Requirements, Data Schema and Architecture

MPOne Idea & Innovation Hackathon 2026 | Technical Track | Problem Statement 6: Resilient & Trustworthy Online Assessment Ecosystem

Document	Contents
Part 1 - PRD	Problem and evidence, goals, users, functional and non-functional requirements, scope, risks, demo plan
Part 2 - Architecture	Layered design, components, data flow, fallback and failover matrix, open-source stack, deployment
Part 3 - Data schema	Event envelope, database tables, audit hash chain, configuration file, API surface, decision rules
Part 4 - Delivery	Build plan, repository layout, test plan, mapping to the hackathon submission checklist

One-line pitch. A plug-in resilience layer that sits above any online exam platform. It predicts and detects failures, works out exactly which candidates were affected from logs, recommends a fair remedy, keeps candidates informed, and records everything in a tamper-evident audit trail, following Prevention -> Detection -> Response -> Recovery -> Trust.

Part 1 - Product Requirements Document

1. Problem and evidence

Large exams are moving online, but failures still trigger re-exams, cost, and loss of trust. Recent public incidents show the same failure families repeating (sources: news reports and official notices retrieved during research; figures are as reported).

Incident	What failed	Impact	Lesson for design
NEET-PG 2026, Jaipur centres	Internal power supply problem during a computer-based exam	2,445 candidates re-sat on 5 Sept	Centre-level power and infrastructure need live monitoring and a resume path
AIAPGET 2026, Jaipur centre	Power outage about 15 minutes after start	Re-exam for affected candidates; NTA suspended the vendor's empanelment	Vendor accountability needs evidence, not memory
JAMB UTME 2025	Software update not installed correctly at some centres	379,997 candidates in 157 centres re-scheduled	Gate centres on software version before the exam starts
California Bar, Feb 2025	Remote platform failures; alleged capacity mismatch	State returned to in-person testing; lawsuits	Capacity must be tested and staggered, not assumed
SSC CGL 2025	Technical disruptions in computer-based tests	About 2,000 reports; regional offices verified each complaint manually	Identify affected candidates from logs, not self-reports
NEET-UG 2026	Paper leak (a security and trust failure)	Exam cancelled, re-exam for over 2.2 million candidates	Tamper-evident records protect trust in outcomes

Root-cause families: (1) centre infrastructure and power, (2) unsafe software rollouts, (3) capacity and overload, (4) slow manual triage with weak evidence, (5) erosion of trust when decisions cannot be verified.

Gap. Existing exam platforms (for example Digiexam and Inspira) protect the candidate's device and connection with autosave and offline buffers. From the research, no open tool covered fleet-level monitoring plus evidence-based remedy decisions plus a verifiable audit trail. ERCT targets that layer. This is our own analysis, not a claim that nothing similar exists.

2. Goals, non-goals and success metrics

ID	Goal	Demo metric (simulated)
G1	Detect centre-level failures quickly	Outage flagged within 10 s of the first missed heartbeats (heartbeat every 2 s)
G2	Identify affected candidates from logs	100% of injected-fault candidates listed; 0 unaffected candidates listed
G3	Recommend a fair, explainable remedy	Every affected candidate gets remedy + rationale + evidence links
G4	Keep candidates informed	Status page reflects incident within 30 s
G5	Provide verifiable evidence	Audit verify passes on clean log; fails and points to the exact entry after a manual edit
G6	Prevent avoidable failures	Centre on wrong software version is blocked by the readiness gate before the exam

Non-goals: not a full exam platform (no question bank, authoring or grading); not proctoring or anti-cheating; no real vendor integration in the MVP (a simulator adapter stands in); no production-grade multi-region deployment.

3. Users and stories

Persona	Need	Story
Candidate	Know what happened and what happens next	As a candidate, when my centre fails I see a clear status and my remedy without chasing anyone.
Centre coordinator	Fast local visibility	As a coordinator, I see my centre's readiness and live health, and I get alerted when power or network drops.
Exam controller	Decide quickly and fairly	As a controller, I get the affected-candidate list and a recommended remedy with reasons, and can approve or override.
Auditor / regulator	Trust the record	As an auditor, I can verify that logs and decisions were not altered after the fact.

4. Functional requirements

ID	Requirement	Priority	MVP
FR1	Ingest events over HTTP/WebSocket with schema validation and idempotent event_id	Must	Yes
FR2	Durable queue with replay so no events are lost during a backend outage	Must	Yes
FR3	Readiness score per centre (software version, power backup, capacity) and a version gate	Should	Yes (table view)
FR4	Live health view per centre and per session (heartbeats, latency, errors)	Must	Yes
FR5	Rule-based detection: heartbeat gap, centre-wide loss, version mismatch, latency breach	Must	Yes
FR6	Streaming anomaly and drift detection on latency and error rate	Should	Yes (basic)
FR7	Incident creation, classification (power, network, software, capacity, security), severity, timeline	Must	Yes
FR8	Affected-candidate identification from session ledger and events	Must	Yes
FR9	Remedy recommendation (resume, extra time, re-test) with rationale from configurable rules	Must	Yes
FR10	Manual review queue when evidence is weak or rules conflict	Should	Yes
FR11	Candidate status page and admin notice feed	Must	Yes
FR12	Hash-chained audit log for events, incidents and decisions, plus a verify function	Must	Yes
FR13	Exportable audit and incident report (HTML/PDF)	Should	Yes (HTML)
FR14	Session resume with restored remaining time	Should	Simulated
FR15	Post-exam analytics: repeat-failure patterns by centre and vendor	Could	Roadmap
FR16	All thresholds, centres and rules driven by a config file	Must	Yes
FR17	Simulator with fault injection (power loss, network drop, wrong version, overload)	Must	Yes

5. Non-functional requirements

- **Reliability:** at-least-once delivery plus idempotent processing, so a replay never creates duplicate incidents or decisions.
- **Integrity:** every audit entry links to the previous hash; any edit breaks verification from that entry onward.
- **Explainability:** each recommendation stores the rule that fired and the evidence rows it used.
- **Privacy:** use pseudonymous candidate IDs in events; keep names and personal data out of the event stream; role-based access for views.
- **Portability:** everything runs from one Docker Compose file on a laptop; no cloud account needed for the demo.
- **Observability:** the platform exports its own metrics to Prometheus and is watched by an independent Uptime Kuma instance.

6. Scope and phasing

Phase	Scope
MVP (submission)	Simulator, ingestion, durable queue, detection (rules + basic River), session ledger, incident manager, impact and remedy engine, status page, dashboard, hash-chained audit with verify, config file, HTML report
Phase 2	Real adapters (for example a Moodle or TAO plug-in), automated failover to backup seats, pre-exam capacity load test with k6, immudb as audit anchor, role-based login
Roadmap	Predictive failure scoring from history, repeat-failure analytics by vendor, multi-region streams, regulator export formats

7. Risks and mitigations

Risk	Mitigation
Very short build time	Freeze scope to the MVP list; simulate the client and centre agent; build one end-to-end incident first
Live demo fails on stage	Record a backup demo video and keep a seeded dataset; run everything from one compose file
Originality rule (work must be created in the hackathon window)	Start a new repository; reuse no prior project code; disclose all open-source libraries, APIs and AI assistants
Tool details change or licences differ	Confirm each licence before submission; keep the stack swappable behind small interfaces
Judges see it as only a dashboard	Lead the demo with the remedy engine and audit verify, the differentiators

8. Hackathon compliance and judging map

Judging criterion (technical track)	Weight	How ERCT addresses it
Innovation and originality	20%	Evidence-based affected-candidate identification and remedy decision support, not just monitoring
Prototype / MVP	20%	Working end-to-end incident demo with fault injection
Problem understanding	15%	Real incident evidence table and root-cause families
Technical feasibility	15%	Open-source stack, Docker Compose, modular services
Impact on higher education / governance	15%	Fewer re-exams, faster decisions, verifiable records for public exams
Scalability and sustainability	10%	Stream-based design, config-driven, adapter model, open licences
Presentation and demo	5%	Five-minute scripted incident walkthrough

Rules to respect: submit between 15 and 30 September; the track cannot change after submission and no edits are allowed afterwards; disclose open-source use and AI assistants; every member must be able to explain and demo the code; a 5 to 10 minute pitch deck of at most 10 slides; working prototype, repository link and tech stack.

9. Demo script (5 minutes)

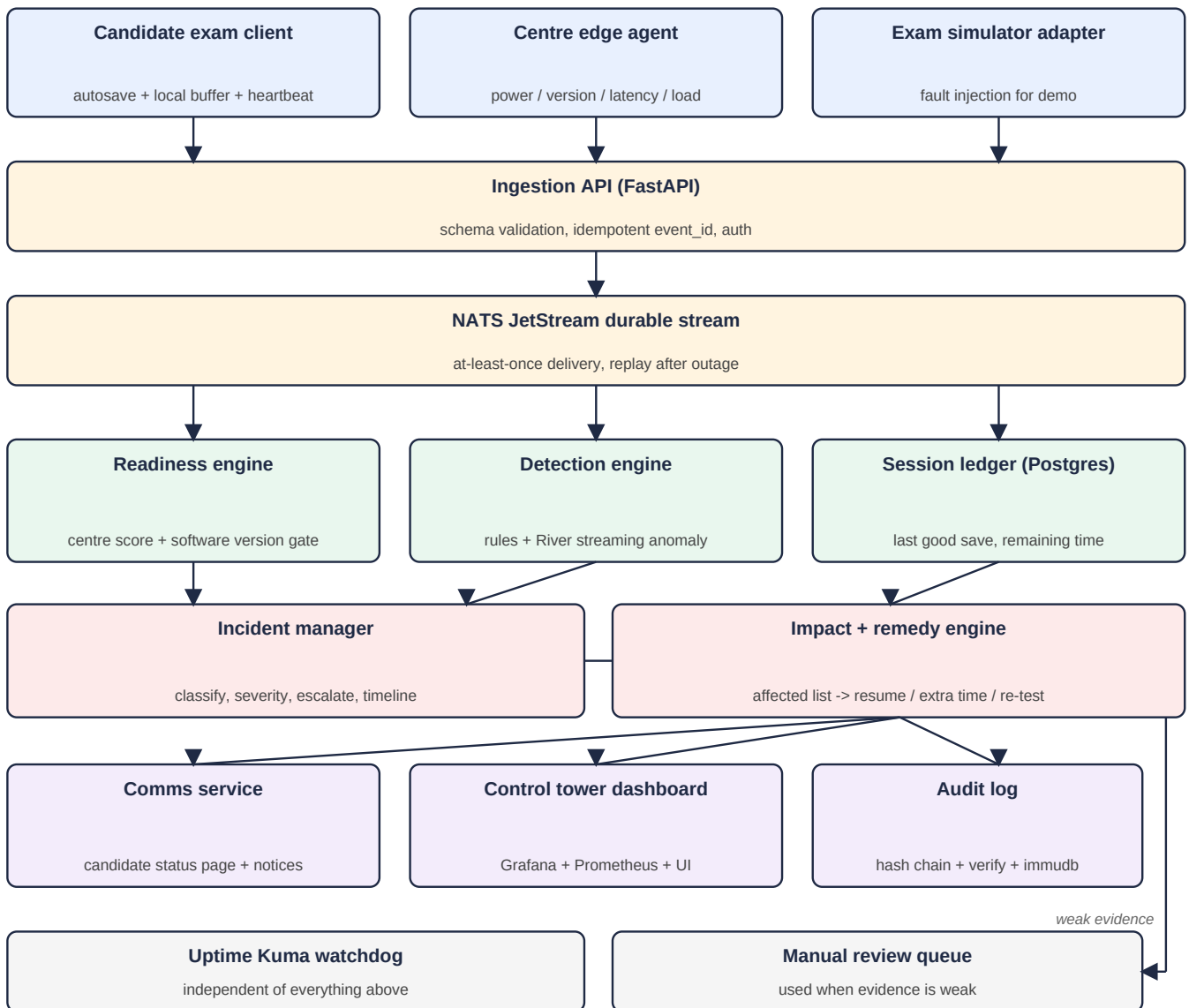
Step	Action	What the judge sees
1	Start 5 simulated centres with about 40 candidates each	Control tower shows all centres healthy; readiness table
2	Show readiness gate: Centre C runs an old version	Centre C blocked before start; alert raised
3	Start exam; inject power loss at Centre B	Heartbeats stop; incident opens within seconds
4	Open the incident	Classified as power, severity high; affected candidates listed with lost time and last good save
5	Remedy engine runs	Recommended: resume with restored time or extra time; weak-evidence cases go to review queue
6	Open candidate status page	Candidate sees what happened and the remedy
7	Restart the ingestion service mid-run	Queued events replay; no duplicates
8	Verify audit, then tamper with one row	Verify passes, then fails at the exact edited entry

Part 2 - Architecture

1. Design principle

ERCT is a **layer**, not a replacement exam system. Any exam platform can feed it through a small adapter that emits the common event format. The demo adapter is a simulator. Every module talks through events, and all parameters come from one configuration file, so the same code handles different centre counts, thresholds and rules.

2. System diagram



Blue = sources, orange = transport, green = analysis, red = decision, purple = outputs, grey = safety nets.

3. Components

Component	Responsibility	Suggested tech
Candidate client (mock)	Generates answers, heartbeats, autosaves; keeps a local buffer and syncs on reconnect	Small web client with PouchDB, or a scripted client in the simulator
Centre edge agent	Reports power state, software version, latency and load; buffers if the uplink drops	Python agent
Ingestion API	Validates events, enforces idempotent event_id, publishes to the stream	FastAPI
Durable stream	Persists events, redelivers unacknowledged ones, allows replay	NATS JetStream
Readiness engine	Scores centres before the exam and enforces the software version gate	Python service
Detection engine	Rules for heartbeat gaps and centre-wide loss; streaming anomaly and drift on latency and errors	Python + River
Session ledger	Latest state per session: last good save, remaining time, state	PostgreSQL
Incident manager	Groups signals into incidents, classifies type and severity, escalates, keeps a timeline	Python service
Impact + remedy engine	Computes affected candidates and recommends a remedy using configurable rules; sends weak cases to review	Python service
Comms service	Candidate status page and admin notices	FastAPI + simple web UI
Dashboard	Live centre map, incident list, metrics	Grafana + Prometheus + small web UI
Audit log	Append-only hash chain of events, incidents and decisions; verify endpoint; optional anchor in immudb	Postgres table + Python; immudb optional
Watchdog	Independent uptime checks and alerts for the platform itself	Uptime Kuma

4. Main data flow

- Sources send events to the Ingestion API. Duplicates are dropped by event_id.
- Valid events go to the durable stream. Consumers acknowledge only after processing, so unacknowledged events are redelivered.
- The session ledger updates from heartbeats and saves. Detection reads the same stream in parallel.
- Detection raises signals. The incident manager merges signals into one incident per centre and cause, and sets severity.
- The remedy engine joins the incident window with the session ledger to list affected candidates and compute lost time and lost answers.
- Each recommendation carries the rule, the evidence and a confidence flag. Low-confidence cases enter the review queue.
- Comms publishes candidate and admin messages. Every event, incident and decision is appended to the audit chain.

5. Fallback and failover matrix

What happens when the normal path breaks. Each row names the failure, the fallback, and how the demo shows it.

Failure	Fallback design	Tooling	Demo
Candidate loses connection	Answers stay in a local buffer and sync on reconnect; server keeps last good save	Local buffer (PouchDB/CouchDB style sync)	Simulated
Centre power or network dies	Session state is held server-side; candidate resumes elsewhere with remaining time restored	Session ledger	Yes
Ingestion or backend down	Store-and-forward: agents retry; stream replays after recovery; idempotent events avoid duplicates	NATS JetStream	Yes (step 7)
Conflicting data after resync	Deterministic rule: latest server timestamp wins per question; all revisions kept in the audit log	Postgres + audit chain	Simulated
Bad software rollout	Version gate blocks the centre before start and raises an alert	Readiness engine	Yes (step 2)
Overload / capacity limit	Staggered slots and admission control; pre-exam load test	k6 (Phase 2)	Roadmap
Monitoring itself fails	Independent watchdog raises out-of-band alerts	Uptime Kuma	Yes

Failure	Fallback design	Tooling	Demo
Weak or missing evidence	No automatic remedy; route to human review with the partial evidence attached	Review queue	Yes
Log tampering	Hash chain breaks at the edited entry; optional anchor in an immutable database	Hash chain, immudb	Yes (step 8)

6. Open-source stack

Tool	Role	Licence / status note
immudb	Immutable, Merkle-tree audit anchor	Apache-2.0 (checked)
PouchDB / CouchDB	Offline-first local buffer and sync pattern	Apache-2.0 (checked)
River	Streaming anomaly and drift detection in Python	BSD 3-clause (checked)
NATS JetStream	Durable stream, at-least-once, replay	Confirm licence before submission
Prometheus, Alertmanager, Grafana	Metrics, alerting, dashboards	Confirm licences (Grafana's licence differs from Prometheus)
Uptime Kuma	Independent watchdog and simple status page	Confirm licence
k6	Load and capacity testing (Phase 2)	Confirm licence
FastAPI, PostgreSQL, Docker Compose	API, storage, packaging	Confirm licences
Safe Exam Browser, Moodle, TAO	Optional adapters and reference exam platforms (not built in MVP)	SEB is Mozilla Public Licence (checked)

Disclose every item above that ends up in the submission, with attribution, as the hackathon rules require.

7. Deployment

```
docker compose up
  nats              (JetStream enabled)
  postgres
  api               FastAPI: ingestion, incidents, remedy, comms, audit
  workers           readiness, detection, incident, remedy consumers
  simulator         centres + candidates + fault injection (reads config.yaml)
  prometheus + grafana
  uptime-kuma
```

Security notes: API keys per centre agent, role-based views (candidate, coordinator, controller, auditor), TLS in any non-local deployment, pseudonymous candidate IDs in events, and no personal data in the audit chain payloads.

Part 3 - Data Schema

1. Event envelope (JSON)

Every source emits this shape. The event_id is a UUID generated at the source and is the idempotency key.

```
{
  "event_id":      "8f1c2e6a-3b7d-4c1a-9d0e-5a2b7c9e1f34",
  "schema_ver":    1,
  "ts":            "2026-10-09T10:20:03.120Z",          // source clock, UTC
  "exam_id":       "EX-2026-PS6-01",
  "centre_id":     "C-BPL-02",
  "candidate_id":  "CAND-000417",                      // null for centre-level events
  "session_id":    "SES-000417-1",                     // null for centre-level events
  "seq":           1287,                                // per-source monotonic counter
  "type":          "HEARTBEAT",
  "severity":      "info",                              // info | warn | error | critical
  "payload":       { "latency_ms": 84, "remaining_s": 5120 }
}
```

Event type	Level	Payload fields
SESSION_STARTED / SESSION_SUBMITTED	Session	duration_s
HEARTBEAT	Session	latency_ms, remaining_s
ANSWER_SAVED	Session	question_id, answer_hash, saved_seq
POWER_LOSS / POWER_RESTORED	Centre	source (grid, ups, generator), backup_minutes
NETWORK_DOWN / NETWORK_UP	Centre	uplink, duration_s (on restore)
LATENCY_SAMPLE	Centre	p50_ms, p95_ms, error_rate
SOFTWARE_VERSION_REPORT	Centre	version, required_version
CAPACITY_SAMPLE	Centre	active_sessions, max_sessions, cpu_pct
CRASH / LOGIN_FAIL	Session	component, error_code

2. Database tables (PostgreSQL)

```
CREATE TABLE exams (
  exam_id TEXT PRIMARY KEY, name TEXT NOT NULL, start_at TIMESTAMPTZ NOT NULL,
  duration_min INT NOT NULL, required_version TEXT NOT NULL, config_hash TEXT NOT NULL);

CREATE TABLE centres (
  centre_id TEXT PRIMARY KEY, name TEXT NOT NULL, city TEXT, vendor TEXT,
  capacity INT NOT NULL, power_backup BOOLEAN NOT NULL DEFAULT FALSE,
  backup_minutes INT DEFAULT 0, software_version TEXT, status TEXT NOT NULL DEFAULT 'ready');

CREATE TABLE candidates (
  candidate_id TEXT PRIMARY KEY, exam_id TEXT REFERENCES exams, centre_id TEXT REFERENCES centres,
  seat_no TEXT, status TEXT NOT NULL DEFAULT 'registered'); -- no names or contact data

CREATE TABLE sessions (
  session_id TEXT PRIMARY KEY, candidate_id TEXT REFERENCES candidates,
  exam_id TEXT REFERENCES exams, centre_id TEXT REFERENCES centres,
  started_at TIMESTAMPTZ, last_heartbeat_at TIMESTAMPTZ, last_saved_seq BIGINT DEFAULT 0,
  answers_saved INT DEFAULT 0, remaining_s INT,
  state TEXT NOT NULL DEFAULT 'active'); -- active|interrupted|resumed|submitted|under_review

CREATE TABLE events (
  event_id UUID PRIMARY KEY, ts TIMESTAMPTZ NOT NULL, ingested_at TIMESTAMPTZ DEFAULT now(),
  exam_id TEXT, centre_id TEXT, candidate_id TEXT, session_id TEXT,
  seq BIGINT, type TEXT NOT NULL, severity TEXT NOT NULL, payload JSONB);
CREATE INDEX ON events (centre_id, ts); CREATE INDEX ON events (session_id, ts);

CREATE TABLE readiness_checks (
  check_id BIGSERIAL PRIMARY KEY, exam_id TEXT, centre_id TEXT, score INT NOT NULL,
  passed BOOLEAN NOT NULL, checks JSONB NOT NULL, checked_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE incidents (
  incident_id TEXT PRIMARY KEY, exam_id TEXT, centre_id TEXT,
  type TEXT NOT NULL, -- power|network|software|capacity|security
  severity TEXT NOT NULL, status TEXT NOT NULL DEFAULT 'open',
  detected_at TIMESTAMPTZ NOT NULL, window_start TIMESTAMPTZ, window_end TIMESTAMPTZ,
  resolved_at TIMESTAMPTZ, detection_rule TEXT, evidence JSONB);

CREATE TABLE incident_impacts (
  incident_id TEXT REFERENCES incidents, candidate_id TEXT REFERENCES candidates,
  session_id TEXT, lost_seconds INT, unsaved_answers INT, last_good_seq BIGINT,
  evidence_quality TEXT, -- strong|partial|missing
  remedy_recommended TEXT, -- resume|extra_time|retest|manual_review
  extra_seconds INT, rationale TEXT, rule_id TEXT,
  PRIMARY KEY (incident_id, candidate_id));

CREATE TABLE decisions (
  decision_id BIGSERIAL PRIMARY KEY, incident_id TEXT, candidate_id TEXT,
  remedy TEXT NOT NULL, mode TEXT NOT NULL, -- auto|approved|overridden
  decided_by TEXT, reason TEXT, created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE review_queue (
  review_id BIGSERIAL PRIMARY KEY, incident_id TEXT, candidate_id TEXT, reason TEXT,
  status TEXT DEFAULT 'pending', assignee TEXT, created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE notices (
  notice_id BIGSERIAL PRIMARY KEY, audience TEXT NOT NULL, -- candidate|centre|admin
  target_id TEXT, incident_id TEXT, message TEXT NOT NULL, created_at TIMESTAMPTZ DEFAULT now());

CREATE TABLE audit_log (
  seq BIGSERIAL PRIMARY KEY, ts TIMESTAMPTZ NOT NULL DEFAULT now(),
  entry_type TEXT NOT NULL, -- event|incident|impact|decision|notice|config
  ref_id TEXT, payload JSONB NOT NULL, prev_hash CHAR(64) NOT NULL, entry_hash CHAR(64) NOT NULL);
```

3. Audit hash chain

```
entry_hash = SHA256( prev_hash || seq || ts_iso || entry_type || ref_id || canonical_json(payload) )
genesis prev_hash = 64 zeros; canonical_json = sorted keys, no whitespace, UTF-8
```

```
verify(): walk entries in seq order, recompute each hash, compare with stored entry_hash and
         check that each prev_hash equals the previous entry_hash. Return first mismatching seq.
anchor(): every N entries (or on incident close) write the head hash to immudb (optional) and
         print it in the report, so the head can be checked outside the database.
```

4. Configuration file (config.yaml)

```
exam:
  id: EX-2026-PS6-01
  duration_min: 120
  required_version: "4.2.1"
simulation:
  centres: 5
  candidates_per_centre: 40
  heartbeat_interval_s: 2
  faults:
    - { centre: C-BPL-02, type: power_loss, at_s: 300, duration_s: 240 }
    - { centre: C-BPL-03, type: wrong_version }
readiness:
  min_score: 70
  weights: { version: 40, power_backup: 30, capacity: 30 }
detection:
  heartbeat_gap_s: 6
  centre_loss_fraction: 0.6          # share of sessions silent within window to call a centre outage
  centre_loss_window_s: 10
  latency_p95_ms: 1500
  anomaly: { enabled: true, threshold: 0.9 }
remedy_rules:
  - { id: R1, if: "lost_s <= 300 and unsaved_answers <= 2 and quality == strong", then: resume }
  - { id: R2, if: "lost_s > 300 and quality == strong", then: extra_time, extra: "lost_s + 120" }
  - { id: R3, if: "quality != strong", then: manual_review }
  - { id: R4, if: "centre_affected_fraction >= 0.5 and integrity_flag", then: retest_recommended }
comms:
  candidate_message_delay_s: 30
audit:
  anchor_every: 200
  immudb: { enabled: false }
```

5. API surface

Method and path	Purpose
POST /v1/events	Ingest one or a batch of events (idempotent by event_id)
GET /v1/centres	Centre list with readiness score and live health
GET /v1/centres/{id}/readiness	Readiness check details and gate result
GET /v1/incidents	Open and closed incidents with severity and type
GET /v1/incidents/{id}	Incident timeline and evidence
GET /v1/incidents/{id}/impact	Affected candidates, lost time, unsaved answers, recommended remedy and rationale
POST /v1/decisions	Approve or override a recommendation (records who and why)
GET /v1/review-queue	Cases waiting for human review
GET /v1/status/{candidate_id}	Candidate-facing status and remedy
GET /v1/audit/verify	Verify the whole chain; returns ok or the first failing seq
GET /v1/audit/report/{exam_id}	Export the audit and incident report (HTML)

6. Decision rules (defaults)

Rule	Condition	Recommendation	Why
R1	Short interruption, at most 2 unsaved answers, strong evidence	Resume with restored remaining time	Minimal harm; avoids a re-exam
R2	Longer interruption, strong evidence	Extra time = lost time + buffer	Compensates exactly the lost time
R3	Evidence partial or missing	Manual review	Do not auto-decide without proof
R4	Half or more of a centre affected and an integrity flag is set	Recommend centre re-test to the controller	Decision support for a whole-centre remedy; a human decides

Fairness check: after remedies are computed, compare average extra time and outcomes across centres and flag large gaps to the controller before approval.

Part 4 - Delivery Plan

1. Repository layout (new repository created for this hackathon)

```
erct/  
  config.yaml  
  docker-compose.yml  
  simulator/  
    api/  
      centres, candidates, fault injection, mock client  
      FastAPI: ingestion, incidents, remedy, comms, audit endpoints  
  workers/  
    readiness, detection, incident, remedy consumers  
  audit/  
    hash chain, verify, report generator, optional immudb anchor  
  web/  
    control tower UI, candidate status page  
  grafana/  
    provisioned dashboards; prometheus/ scrape config  
  docs/  
    architecture, schema, README, ATTRIBUTIONS.md  
  tests/  
    unit tests for rules, hash chain, idempotency
```

2. Build order (priority first)

Order	Deliverable	Done when
1	Event schema, API, stream, Postgres tables	Events flow from a script into the database
2	Simulator with 5 centres and fault injection	Power loss stops heartbeats for one centre
3	Detection rules and incident manager	Incident opens within 10 s of the fault
4	Impact and remedy engine	Affected list and remedies match the injected fault
5	Audit chain and verify	Verify passes; a manual edit is caught
6	Dashboard and candidate status page	Both update live during the demo
7	Readiness gate and River anomaly	Wrong-version centre blocked; latency spike flagged
8	Replay test, report export, demo video	Restart mid-run causes no duplicates; video recorded

3. Test plan

- **Idempotency:** send the same event twice; expect one stored row and one audit entry.
- **Replay:** stop the API for 30 s during the run; expect all events after restart and no duplicate incidents.
- **Accuracy:** inject a fault for a known set of candidates; expect the affected list to equal that set.
- **Integrity:** edit one payload directly in the database; expect verify to fail at that seq.
- **Rules:** unit tests for each remedy rule with boundary values.

4. Mapping to the required submission documents

#	Required item (from the hackathon manual)	Where it comes from
1	Solution synopsis / executive summary	Cover page pitch and Part 1 sections 1 to 2
2	Solution presentation (PDF, max 10 slides)	Slides built from this document; use the diagram and demo script
3	Problem statement and proposed solution	Part 1 section 1 and Part 2 section 1
4	Innovation and differentiation note	Part 1 gap statement; remedy engine and audit verify
5	Impact and benefits document	Part 1 goals and judging map
6	Implementation / feasibility plan	Part 4 build order and Part 2 stack
7	Technology architecture / technical approach	Part 2 (required for technical track)
8	Prototype / demo / proof of concept	Working build plus recorded demo video
9	Code repository URL	New repository (required for technical track)
10	Why should this solution be selected?	Differentiators: log-based impact, explainable remedies, verifiable audit trail

5. Attribution and disclosure

Keep an ATTRIBUTIONS.md in the repository listing every open-source library, public API, starter template and AI coding assistant used, with licence. The hackathon rules require this disclosure, and every team member should be able to explain and demo any submitted code.